Graph-Preprocessed ILP Scheduling for DFG Placement

Junchi Wu
Student ID: 2501111908
Peking University
Beijing, China

Feiyang Wu
Student ID: 2501111907
Peking University
Beijing, China

***Abstract*—We study placement and scheduling of data-flow graphs on a linear slice-based architecture with streaming communication. The implementation provides a direct integer-linear-programming baseline and a graph-preprocessed mode that tightens issue-time windows and prunes resource-pair constraints before constructing the same domain model. A post-solve checker independently validates placement, dependency timing, full-duration same-slice non-overlap, stream routing, and makespan semantics. On solved existing workloads, graph preprocessing preserves makespan and reduces constraints by 5.7%–21.8%. Generated workloads up to 100 operations produce feasible or optimal schedules within a 60 s budget, with every solved result passing the domain checker. The results support correctness and model-size reduction, while showing that solve-time speedup remains workload-dependent.**

***Index Terms*—integer linear programming, DFG scheduling, graph preprocessing, placement, verification**

I. Introduction

Modern accelerator software often fails in the space between a clean graph abstraction and a physical machine. A DFG says that one operation depends on another, but the hardware asks sharper questions: where does the value live, how far must it travel, which stream carries it, and whether two operations silently collide on the same slice. A scheduler that ignores these questions can look correct at graph level while producing a schedule that cannot run.

The central idea of this project is to keep the physical constraints explicit while using graph structure to make the ILP smaller. We do not replace the solver with a heuristic scheduler. Instead, the graph pass computes facts that are already implied by dependencies, such as earliest/latest issue windows and operation-pair comparability, and passes those facts into a shared ILP model. This design makes the baseline and optimized mode comparable: both solve the same architectural problem, but the graph mode gives the solver less irrelevant work.

This report makes three contributions. First, it formulates a baseline ILP for joint placement, timing, stream routing, and full-duration same-slice non-overlap. Second, it adds a graph-preprocessed mode that tightens issue domains and safely prunes resource pairs without changing the domain semantics. Third, it implements an independent domain-level checker and uses it to validate all feasible or optimal experimental results.

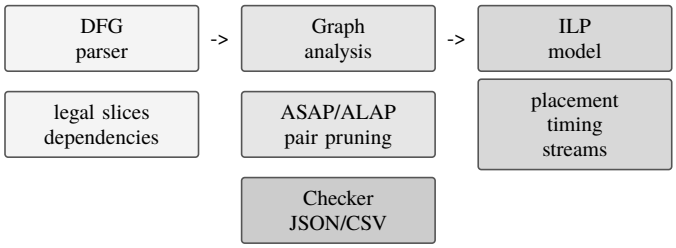

Fig. 1. Solver flow. Graph preprocessing narrows issue windows and removes safe operation-pair constraints before the shared ILP model is solved and checked.

II. Motivation and Background

Specialized data-flow accelerators expose high throughput only when computation, memory movement, and inter-slice streams are scheduled together. In this project, each DFG operation is assigned to a hardware slice and issue cycle, while each edge is routed through a directional stream. A direct ILP formulation is attractive because it states the architecture constraints exactly, but it scales poorly as every pair of operations and every pair of streams can introduce binary decisions.

The target abstraction is deliberately small but captures the essential coupling in the placement problem. Compute operations often have narrow legal slice sets, memory operations may live on banks away from the vector slice, and edge latency depends on physical distance. A placement decision therefore changes both local resource use and communication timing. Treating placement first and scheduling later can miss feasible schedules or accept schedules that violate the streaming model; treating all decisions together gives correctness but increases the size of the mixed-integer search.

The project therefore separates two concerns. The baseline keeps the full ILP model and serves as the reference. The graph-preprocessed mode first analyzes the DFG topology to compute issue-time bounds and identify operation pairs whose relative order is already guaranteed by dependency timing. Both modes then use the same placement, timing, stream, and full-duration non-overlap semantics. This keeps the comparison fair: preprocessing may reduce model size, but it does not relax correctness.

III. Problem Formulation

Given a DFG $G = (V, E)$, each operation $i \in V$ has a kind, a duration d_i, and an optional feasible slice set S_i. The solver chooses integer issue time t_i, slice position p_i, and finish time $f_i = t_i + d_i$. For each edge (i, j), it also chooses a stream id, direction, route interval, and phase.

The hardware is modeled as a one-dimensional slice line from $p_{\min}$ to $p_{\max}$. Binary placement variables $y_{i,p}$ select exactly one legal slice for each operation. The position variable is then linked to the selected slice by $p_i = \sum_p p y_{i,p}$. This representation keeps operation legality local while still exposing positions to timing and routing constraints.

For every edge $e = (i, j)$, the model stores route bounds l_e and h_e, a phase φ_e, a direction bit, and a one-hot stream id. The route interval is the closed interval between p_i and p_j, with east/west direction selected according to the sign of $p_j - p_i$. The phase represents the time at which the transmitted value enters its route, normalized by the source issue time and source position. Pairwise stream constraints then compare two edges only when they share direction, stream id, and an overlapping route interval.

The objective is to minimize makespan:

$$\min M \quad \text{s.t.} \quad M \geq t_i + d_i, \forall i \in V. \tag{1}$$

For compute-involved edges, the implementation enforces exact timing with stream latency k and physical slice distance:

$$t_j - t_i = d_i + k + |p_i - p_j|. \tag{2}$$

For memory-to-memory edges, the same expression is a lower bound, allowing later issue times when memory movement is not tied to compute-cycle equality. The full-duration resource constraint forbids same-slice interval overlap:

$$p_i = p_j \Rightarrow \big(t_i + d_i \leq t_j\big) \vee \big(t_j + d_j \leq t_i\big). \tag{3}$$

The implementation encodes this disjunction with one binary order variable $b_{i,j}$ per active operation pair and per-slice guards from the placement selectors:

$$t_i + d_i \leq t_j + M\big(3 - y_{i,p} - y_{j,p} - b_{i,j}\big), \tag{4}$$

$$t_j + d_j \leq t_i + M\big(2 - y_{i,p} - y_{j,p} + b_{i,j}\big). \tag{5}$$

When both operations choose slice p, exactly one ordering is enforced; otherwise the Big-M terms relax the row. This is stronger than forbidding only equal issue cycles: two operations of duration greater than one may issue at different cycles and still overlap. The stricter form is necessary for a capacity-one slice model.

Stream constraints select one direction and one stream id for every edge, bind the route interval to source/destination positions, and require non-conflicting phase order for edges sharing stream id, direction, and route overlap. These constraints are exported to JSON, then checked independently by a domain verifier.

IV. Algorithms

A. Baseline ILP

The baseline constructs the model over all operations and all operation pairs. It creates placement selectors, issue variables, absolute position-distance variables for timing, binary interval-order variables for same-slice non-overlap, and stream-pair variables for route conflicts. This mode is the correctness reference and uses the same objective and domain checker as the graph mode.

The baseline is intentionally not optimized by graph reachability. It is useful because every pair of operations is considered for resource conflicts, so it exposes the full cost of the direct ILP. It also gives a clean comparison point for preprocessing: if graph mode changes makespan or violates the domain checker, the change is a correctness bug rather than a modeling difference.

The largest baseline growth terms are quadratic. For $|V|$ operations, the full resource model has $|V|\frac{|V|-1}{2}$ interval-order candidates, and for $|E|$ edges, stream conflict modeling considers $|E|\frac{|E|-1}{2}$ edge pairs. The implementation therefore measures variable count, constraint count, active operation pairs, and pruned operation pairs in every JSON result.

B. Graph Preprocessing

Graph preprocessing performs a topological traversal to compute ASAP and ALAP issue-time bounds. For each edge, the minimum possible latency is $d_i + k + \min|p_i - p_j|$ over legal slices. Forward propagation gives earliest times; reverse propagation gives latest times under the global issue horizon. The same reachability analysis classifies operation pairs as comparable or incomparable.

The bounds are conservative because they use minimum feasible physical distance rather than a selected placement. They cannot remove a valid solution; they only exclude issue times that are impossible even under the best slice choices. In practice this reduces issue domains for downstream operations and can also make the branch-and-bound search easier because fewer integer values remain available.

Reachability is computed over the DAG in reverse topological order. If i reaches j, and the dependency timing is exact or otherwise proves separation, then an explicit non-overlap row for (i, j) is redundant: the schedule already orders the intervals. If neither reaches the other, the pair is incomparable and may overlap unless the ILP chooses different slices or an explicit order.

Comparable pairs are pruned from full-duration resource constraints only when dependency timing proves they cannot overlap. Memory-memory direct edges are retained conservatively because they use lower-bound rather than exact timing. The resulting active pair set is:

$$P_{\text{graph}} = P_{\text{incomparable}} \cup P_{\text{unsafe comparable}}. \tag{6}$$

This policy is more conservative than simply dropping all reachable pairs. Compute-involved dependencies enforce exact arrival, so the predecessor's finish and communication latency determine the successor's issue. Memory-memory edges, however, allow slack and therefore may not prove interval

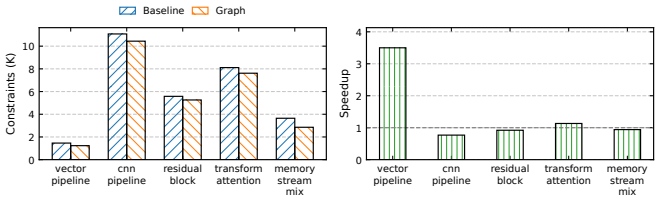

Fig. 2. Existing workload comparison. Graph preprocessing preserves makespan on solved workloads and reduces model constraints; solve-time speedup is workload-dependent.

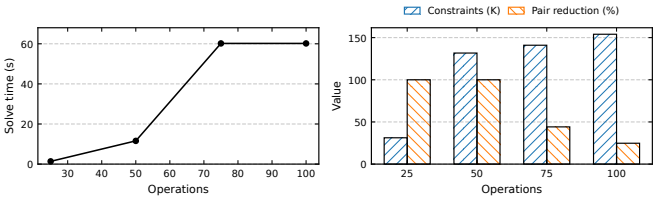

Fig. 3. Generated scalability workloads. The 75- and 100-operation cases reach feasible schedules at the 60 s limit and pass the domain checker.

separation on their own. Keeping these unsafe comparable pairs avoids turning preprocessing into an implicit relaxation.

C. Domain-Level Checker

After an optimal or feasible solve, the checker reconstructs the schedule outside OR-Tools and verifies all domain semantics: legal slices, one assignment per operation, finish-time consistency, makespan coverage, exact or lower-bound edge timing, full-duration same-slice non-overlap, stream id bounds, stream direction, route intervals, phases, arrival timing, and pairwise stream conflicts. A feasible schedule is accepted only when this checker reports zero errors.

This checker is separate from the solver's numerical constraint verifier. The numerical verifier can confirm that linear rows are satisfied, but it does not explain whether the rows jointly match the intended accelerator semantics. The domain checker reports human-readable errors, making it useful for regression tests and for evaluating generated JSON results after the solver has exited.

The checker also protects against modeling drift. For example, if a future optimization accidentally removes a necessary same-slice order, the checker reports the concrete pair of operations and their overlapping intervals. If a stream assignment has the wrong direction or phase, the checker reports the edge and expected route semantics. This makes evaluation artifacts auditable rather than relying only on solver status strings.

V. Experimental Results

Experiments use the regenerated CSV/JSON artifacts in `evaluation/mode_comparison` and `evaluation/scalability`. The time limit is 90 s for existing baseline-vs-graph comparisons and 60 s for generated scalability workloads. Rows marked optimal or feasible all pass the domain checker.

The existing workload set contains vector, CNN-style, residual, transformer-attention, memory-heavy, and legacy tree-reduction graphs. The scalability generator creates deterministic workloads at 25, 50, 75, and 100 operations. For the larger generated cases, a 50-operation dependent pipeline is combined with independent fixed-slice operations so the experiment stresses operation-pair non-overlap without turning stream-pair constraints into the only measured bottleneck.

Fig. 2 and Table I show that the graph mode reduces constraints on all solved existing workloads while preserving makespan. The largest measured solve-time speedup is 3.50x

TABLE I

Existing workloads under full-duration non-overlap. Tree reduce is infeasible in both modes, revealing a legacy workload invalidated by stricter resource semantics.

Workload	Base	Graph	Cons. Red.	Check
Vector	opt	opt	15.6%	pass
CNN	opt	opt	5.7%	pass
Residual	opt	opt	5.7%	pass
Tree reduce	inf	inf	–	–
Attention	opt	opt	6.1%	pass
Memory mix	opt	opt	21.8%	pass

on the vector pipeline, but CNN, residual, and memory-mix workloads are not faster under the stronger model. Thus the supported performance claim is model-size reduction with workload-dependent speed, not universal acceleration.

This distinction is important. Reducing constraints does not guarantee faster branch-and-bound search because the remaining formulation may expose a harder search tree or different LP relaxations. We therefore report speedup as an observed metric rather than as a claim guaranteed by preprocessing. The functional claim is stronger: graph mode preserves the objective value on solved workloads and every accepted schedule passes the same checker.

The infeasible tree-reduction result is also informative. Under the earlier same-issue-cycle conflict model, the workload could place long operations on the same slice with overlapping execution intervals as long as their issue cycles differed. Full-duration non-overlap invalidates that schedule in both baseline and graph modes. We therefore treat this workload as a legacy negative test rather than as performance evidence.

The scalability study in Fig. 3 reaches 100 operations. The 25- and 50-operation cases solve optimally; the 75- and 100-operation cases return feasible schedules at the 60 s budget. Pair reduction remains substantial but decreases for larger generated workloads because the generator intentionally combines a 50-operation dependent pipeline with independent fixed-slice operations to test full-duration non-overlap without making stream-pair constraints dominate the benchmark.

TABLE II

Scalability evidence from graph-preprocessed mode. The 75- and 100-operation workloads return feasible checked schedules at the configured time budget.

Case	Ops	Status	Makespan	Time	Check
Scale-25	25	opt	75	1.37 s	pass
Scale-50	50	opt	149	11.58 s	pass
Scale-75	75	feas.	189	60.18 s	pass
Scale-100	100	feas.	239	60.18 s	pass

Table II gives the exact status and timing for the generated workloads. The feasible rows are still useful because the objective is makespan minimization and the solver may stop at the time limit before proving optimality. The domain checker accepts both optimal and feasible rows only after validating the recovered schedule.

The scalability data should be read as a functional and modeling result rather than a proof of asymptotic efficiency. The current stream formulation still creates pairwise edge-conflict variables, so dense DFGs can become difficult even when operation-pair pruning is effective. The generated cases show that the implemented graph mode can produce checked schedules at the slide target size, while leaving room for future stream-conflict preprocessing.

VI. Limitations and Future Work

The current implementation validates the core scheduling story, but it is not yet a complete compiler backend. The DFG format is intentionally minimal, the hardware model uses a uniform capacity-one slice abstraction, and the generated scalability workloads are controlled rather than sampled from a full application compiler. These choices make the model understandable and testable, but they should be revisited when integrating richer operation types or multi-capacity resources.

The main algorithmic bottleneck is stream conflict modeling. Operation-pair pruning reduces non-overlap constraints, but stream conflicts still scale with edge pairs. A natural extension is a stream-preprocessing pass that proves route disjointness, incompatible directions, or phase separation before variables are created. Another extension is benchmark generation: feasible reduction and branching patterns would make the evaluation less pipeline-oriented while still respecting exact compute timing.

VII. Conclusion

The repository now functionally supports the claimed baseline-plus-graph-preprocessing workflow. It has a direct ILP baseline, a graph-preprocessed mode, a stronger full-duration non-overlap model, and an independent domain checker that validates feasible schedules. The experimental artifacts support correctness, model-size reduction, and feasibility through 100 operations. The only claim that must be qualified is solve-time speedup: preprocessing reduces constraints and can accelerate selected workloads, but the observed speedup depends on workload structure and solver behavior.